Laboratorio No.1 – Semana 14 - Integración servicios en la nube

Nombre: _____ **Carné**: _____

Objetivo: Desarrollar y publicar una aplicación web que permita interactuar con archivos almacenados en Azure Blob Storage, utilizando control de versiones con Git y una canalización CI/CD de Azure DevOps para automatizar el despliegue de la aplicación en Azure App Service

Resultado esperado

- Aplicación web funcional ejecutándose en Azure App Service.
- Conexión entre App Service y Azure Blob Storage.
- Listado y consulta de archivos almacenados en un contenedor Blob.
- Operación de carga de archivos al Blob Storage.
- Repositorio Git con historial de cambios.
- Pipeline de Azure DevOps que compile y despliegue automáticamente la aplicación.
- Evidencias que demuestren el funcionamiento de la solución.

Repositorio recomendado
Utilice como punto de partida el ejemplo oficial de Microsoft/Azure Samples revisado para esta actividad:
GitHub: https://github.com/Azure-Samples/serviceconnector-webapp-storageblob-dotnet
El tutorial oficial utiliza una aplicación web .NET conectada a Azure Blob Storage mediante una identidad administrada y Service Connector. La documentación indica además que la aplicación puede desplegarse en Azure App Service.

Tecnologías

Tecnología	Uso
C# / .NET	Desarrollo de la aplicación web.
Azure Blob Storage	Almacenamiento de archivos.
Azure App Service	Hospedaje de la aplicación.
Git	Control de versiones.
Azure DevOps Pipelines	Integración y despliegue continuo.
Managed Identity / Service Connector	Acceso seguro desde App Service al almacenamiento.

Realización de la actividad

1. Preparar el repositorio
En el repositorio brindado arriba, realice un fork del repositorio base o copie el proyecto a un repositorio propio. El repositorio debe pertenecer al equipo de trabajo. Realice un primer commit identificable

---

2. Revisar la aplicación:
Ejecute la aplicación localmente y revise cómo obtiene acceso al Blob Storage. Identifique el punto del código donde se realiza la lectura/escritura de blobs y documente brevemente su funcionamiento. Visual Studio -> carpeta -> MIA_Semana14 -> git clone [repo url] -> dotnet commands

NOTA: cambia ren .csproj file -> <TargetFramework>net9.0</TargetFramework>

3. Crear DotNet Env

a) En terminal, instalar: **dotnet add package DotNetEnv**
b) Crear .env file en raíz de proyecto, valores ->

AZURE_STORAGEBLOB_RESOURCEENDPOINT=https://miarchivos2026.blob.core.windows.net/
AZURE_STORAGE_CONTAINER_NAME=default

c) Crear .env.production con los mismos valores en el mismo lugar
d) Anadir cambio brindado en program.cs
e) dotnet clean -> dotnet restore -> dotnet build -> dotnet run

4. Crear recursos en Azure

En azure portal site, cree o utilice un Resource Group, una Storage Account con Blob Storage y un contenedor. El contenedor deberá tener al menos tres archivos de prueba, Puede utilizar el creado en semana 13.

5. Conectar App Service con Blob Storage

Configure la conexión entre App Service y Blob Storage mediante Service Connector utilizando identidad administrada. Verifique que la identidad tenga los permisos necesarios sobre los blobs.

a) Azure Portal → **Storage accounts → tu Storage Account → Data storage → Containers**
b) Crear contenedor "default
c) Activa identidad administrada en App Service: **Azure Portal → App Service → Settings → Identity → System assigned: on → Save (web app creada semana 13)**
d) Configurar la conexión entre App Service y Blob Storage: **App Service → Settings → Service Connector → Create → Seleccionar "Storage Blob" → Seleccionar Storage Account [tu_blob_name] → Authentication "System-assigned managed identity" → Review + Create**.
e) Reemplazar storage blob endpoint y container nombre en .env file
Storage Account → Settings → Endpoints → Blob service

6. Probar el acceso al almacenamiento:

Ejecute la aplicación local y compruebe que puede leer los blobs. Realice al menos una operación de carga de archivo y verifique desde Azure Portal que el archivo llegó al contenedor. Visual Studio -> dotnet run

a) Verifique que Azure CLI esté instalado ejecutando en la terminal de VS Code: az --version
b) Inicie sesión en Azure: az login
c) Verifique la cuenta de Azure activa: az account show
d) Verifique que pueda obtener un token para Azure Storage: az account get-access-token --resource https://storage.azure.com/
e) Si se muestra un accessToken la autenticación esta correcta, ejecutar -> dotnet run
f) Si hay error en acceso, ejecutar en terminarl vs code: az role assignment create --assignee "TU_CORREO_DE_AZURE" --role "Storage Blob Data Contributor" --scope "/subscriptions/TU_SUBSCRIPTION_ID/resourceGroups/TU_RESOURCE_GROUP/providers/Microsoft.Storage/storageAccounts/TU_STORAGE"

Obtener info: az account show --query "{subscriptionId:id, user:user.name}" -o table
az storage account show --name "tu_storage" --query "{id:id, resourceGroup:resourceGroup}" -o table

g) Cerrar process ctr +c y logear de nuevo: az logout y az login, Por ultimo dotnet run

Nota: no olvidar subir cambios a repo github fork

---

7. Crear Pipeline Yml

En la raíz de tu fork de GitHub crear archivo “azure-pipelines.yml”
Colocar en el archivo lo siguiente:

```
trigger:
- main

pool:
  vmImage: 'windows-latest'

variables:
  buildConfiguration: 'Release'
  dotnetVersion: '9.x'

  # Completar posteriormente
  azureSubscription: 'NOMBRE-DE-TU-SERVICE-CONNECTION'
  appName: 'NOMBRE-DE-TU-APP-SERVICE'

steps:

# 1. Instalar .NET 9
- task: UseDotNet@2
  displayName: 'Instalar .NET 9 SDK'
  inputs:
    packageType: 'sdk'
    version: '$(dotnetVersion)'

# 2. Restaurar dependencias
- task: DotNetCoreCLI@2
  displayName: 'Restaurar dependencias'
  inputs:
    command: 'restore'
    projects: '**/*.csproj'

# 3. Configurar ambiente de producción
- powershell: |
    Copy-Item ".env.production" ".env" -Force
  displayName: 'Configurar .env de producción'

# 4. Compilar
- task: DotNetCoreCLI@2
  displayName: 'Compilar aplicación'
  inputs:
    command: 'build'
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --no-restore'
```

```
# 5. Publicar
- task: DotNetCoreCLI@2
  displayName: 'Publicar aplicación'
  inputs:
    command: 'publish'
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --output $(Build.ArtifactStagingDirectory)/publish --no-build'
    zipAfterPublish: true

# 6. Desplegar en Azure App Service
- task: AzureWebApp@1
  displayName: 'Desplegar en Azure App Service'
  inputs:
    azureSubscription: '$(azureSubscription)'
    appType: 'webApp'
    appName: '$(appName)'
    package: '$(Build.ArtifactStagingDirectory)/publish/**/*.zip'
```

---

8. Crear pipeline CI/CD

Cree un pipeline YAML que restaure dependencias, compile, publique y despliegue la aplicación en Azure App Service.

a) Conectarse a Azure Devops son misma cuenta que azure portal -> https://aex.dev.azure.com/me

b) Crear pipeline: **En Azure devOps -> Menu izquierda -> pipelines -> Create Pipeline -> Seleccionar Github -> Autoriza Azure DevOps para acceder a GitHub -> seleccionar repositorio que hizo fork -> seleccionar Existing Azure Pipelines YAML file -> Branch: main Path: /azure-pipelines.yml**

c) Probar seleccionando "run"

d) Conectar Azure DevOps con Azure: **Project Settings → Service connections → New service connection -> Azure Resource Manager -> luego, Workload identity federation -> Selecciona tu suscripción de Azure -> y resource group donde estan tus recursos -> Anadir nombre "MIA_Connection" -> Seleccionar "grant access permission to all pipelines" -> Save**

---

8. Conectar Pipeline con App

a) Obtener nombre app: En Azure Portal App Services → tu aplicación → Copia nombre app

b) en github en tu "azure-pipelines.yml" file busca: azureSubscription y appName, cambiar por valores reales.

```
azureSubscription:  TU_Service_Connection_name
appName: Tu _app_name
```

c) Al hacer commit, verificar el pipeline se ejecuta automáticamente

---

9. Validar la solución

Compruebe desde la URL pública que la aplicación funciona y que el acceso al Blob Storage continúa operativo después del despliegue.

a) acceso de la aplicación al Blob: Azure Portal → App Service → Settings → Environment variables → App settings -> colocar AZURE_STORAGEBLOB_RESOURCEENDPOINT
https://tu_storage.blob.core.windows.net/
(si existe obviar)
b) Verificar app: Azure Portal → App Service → OverView → Default Domain → click en URL publica
c) En la app publicada realizar cambios en archivos
d) Verificar que los cambios se representen en storage account: Azure Portal → Storage Account → Storage Browser → Blob containers → Seleccionar su container → Verificar cambios

---

10. Preguntas de analisis

1. **¿Para qué sirve el archivo .env en la aplicación?**
   El archivo .env sirve para cargar variables de entorno durante el desarrollo local, como cadenas de conexión, URLs, nombres de contenedores, claves o configuración específica. Evita hardcodear valores en el código y permite tener distintos entornos, pero no debe subirse a GitHub. En producción se recomienda usar App Settings, Key Vault o identidad administrada en lugar de .env**.**

2. **¿Por qué utilizar Blob Storage y no guardar los archivos dentro del proyecto?**
   Porque el almacenamiento local del App Service es efímero, se pierde al reiniciar o redeploy, no se comparte entre instancias si la app escala y es más difícil de respaldar, auditar y proteger. Blob Storage ofrece persistencia, escalabilidad, acceso desde múltiples instancias, seguridad con RBAC o SAS, lifecycle management, versionado y mejor separación entre cómputo y almacenamiento.

3. **¿Cuál es la diferencia entre el endpoint de Blob Storage y la URL de App Service?**
   El endpoint de Blob Storage es la URL del servicio de almacenamiento, por ejemplo https://micuenta.blob.core.windows.net, y se usa para leer o escribir blobs mediante SDK, REST API o SAS. La URL de App Service es la dirección pública de la aplicación web o API, por ejemplo https://mi-app.azurewebsites.net, y se usa para que los usuarios consuman la aplicación. No son intercambiables: la app recibe la petición en la URL de App Service y luego escribe en el endpoint de Blob Storage o genera una SAS para que el cliente suba directo.

4. **¿Por qué utilizar una identidad administrada en lugar de una Storage Account Key?**
   Porque la Storage Account Key es un secreto global muy poderoso que, si se filtra, compromete toda la cuenta, no permite permisos finos, hay que rotarla manualmente y es difícil de auditar. La identidad administrada permite autenticación sin secretos, permisos finos con RBAC, rotación automática, auditoría en Entra ID y mínimo privilegio. Para subir blobs normalmente se asigna el rol Storage Blob Data Contributor sobre la Storage Account o el contenedor.

5. **¿Qué función cumplen GitHub, Azure DevOps y la Service Connection?**
   GitHub es el repositorio de código fuente, guarda el proyecto, controla versiones, permite Pull Requests y dispara el pipeline al hacer push. Azure DevOps orquesta el CI/CD: restaura dependencias, compila, corre pruebas, publica artefactos y despliega en Azure. La Service Connection es la conexión segura entre Azure DevOps y Azure, y permite que el pipeline despliegue en App Service, Storage, etc., sin guardar credenciales manuales, usando federated credentials, service principal o workload identity.

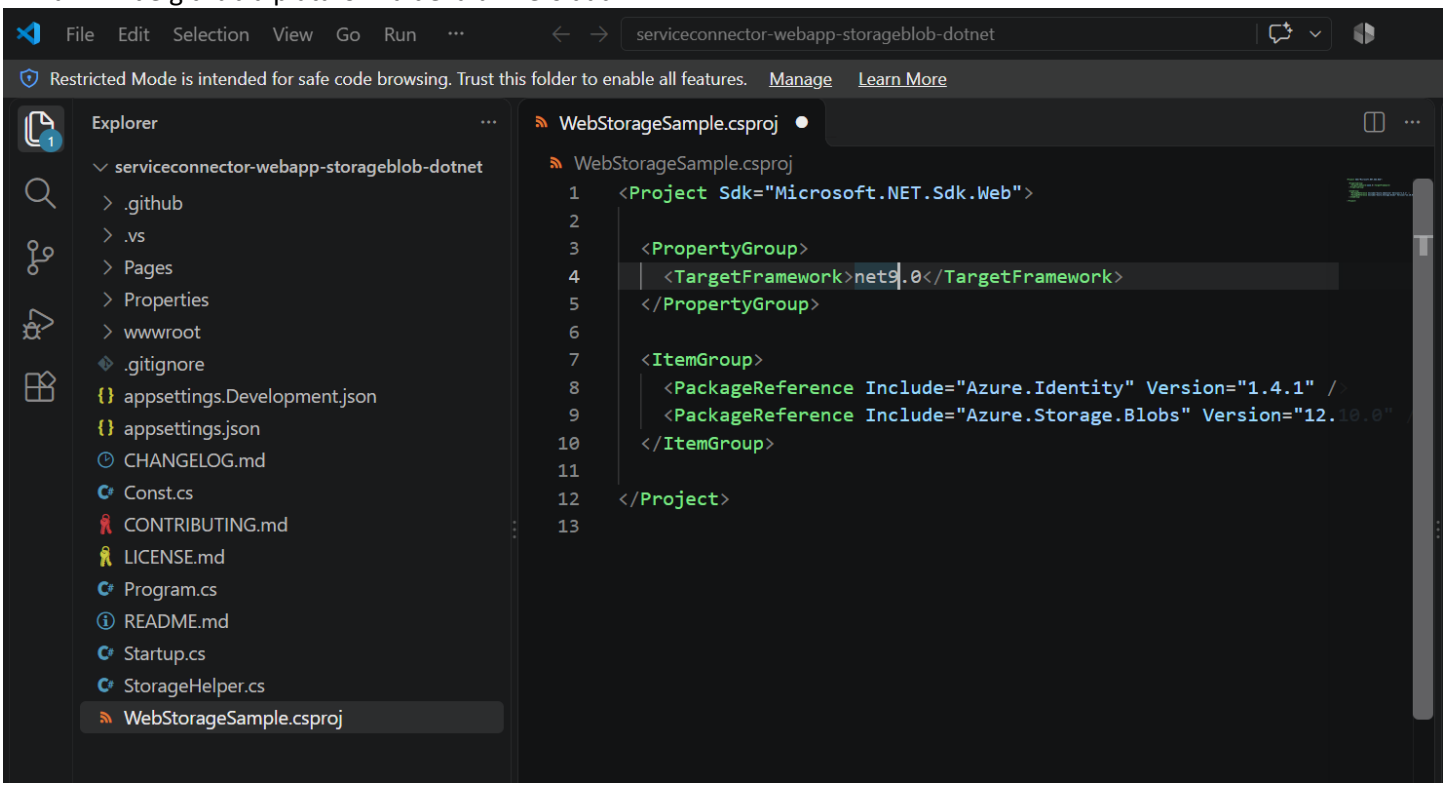
6. **¿Qué ocurre después de hacer push a main?**
   Después de hacer push a main, GitHub recibe el cambio y se dispara un webhook o trigger hacia Azure DevOps. El pipeline inicia, hace checkout del código, restaura dependencias, compila, ejecuta pruebas, publica el artefacto y despliega en el App Service. Luego actualiza App Settings si corresponde, reinicia la app, ejecuta health checks y notifica éxito o fallo. Si todo sale bien, la nueva versión queda desplegada y accesible desde la URL del App Service.

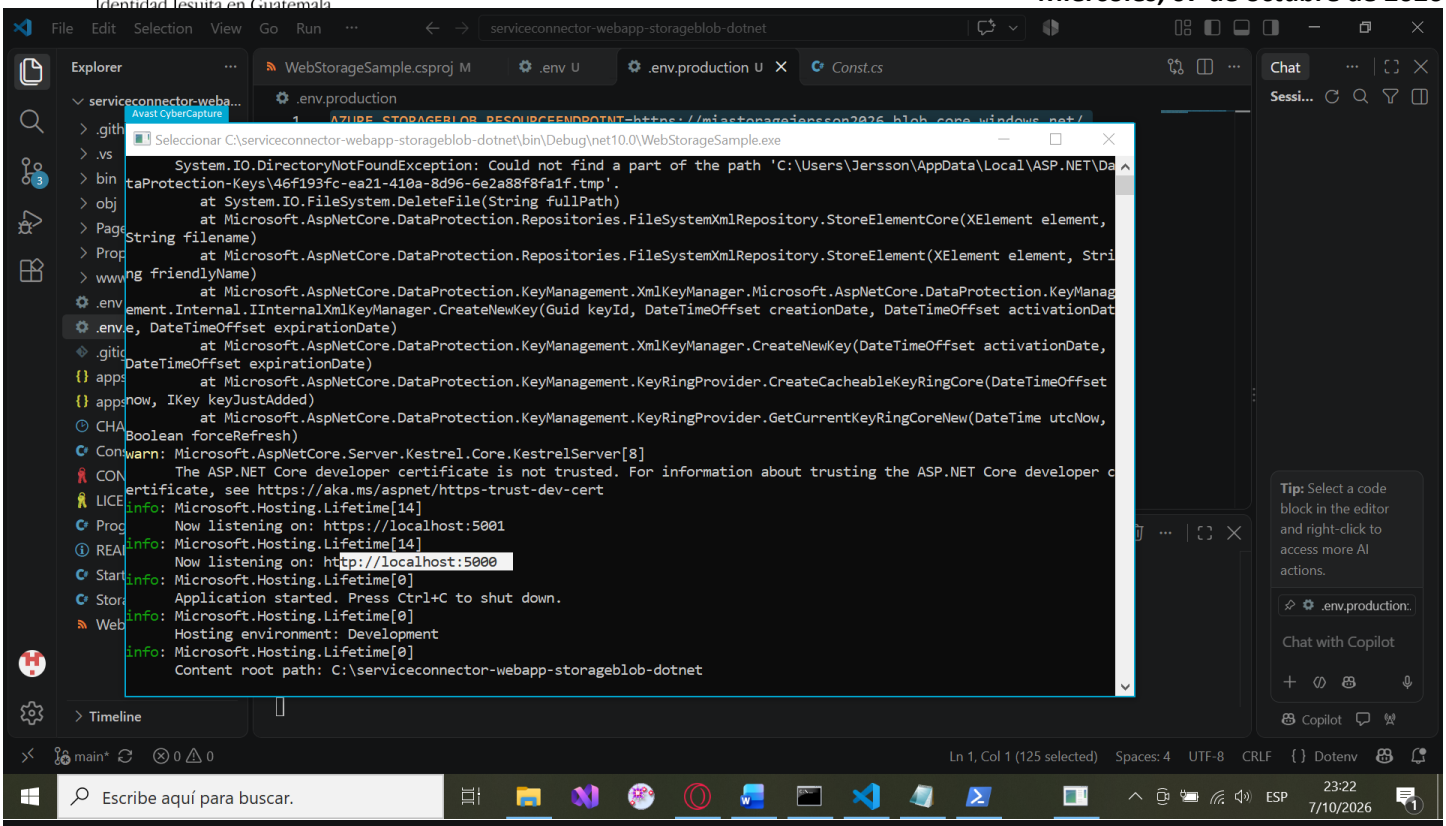
7. **Si el pipeline termina correctamente pero la aplicación recibe un 403 al subir un archivo, ¿qué revisarías?**

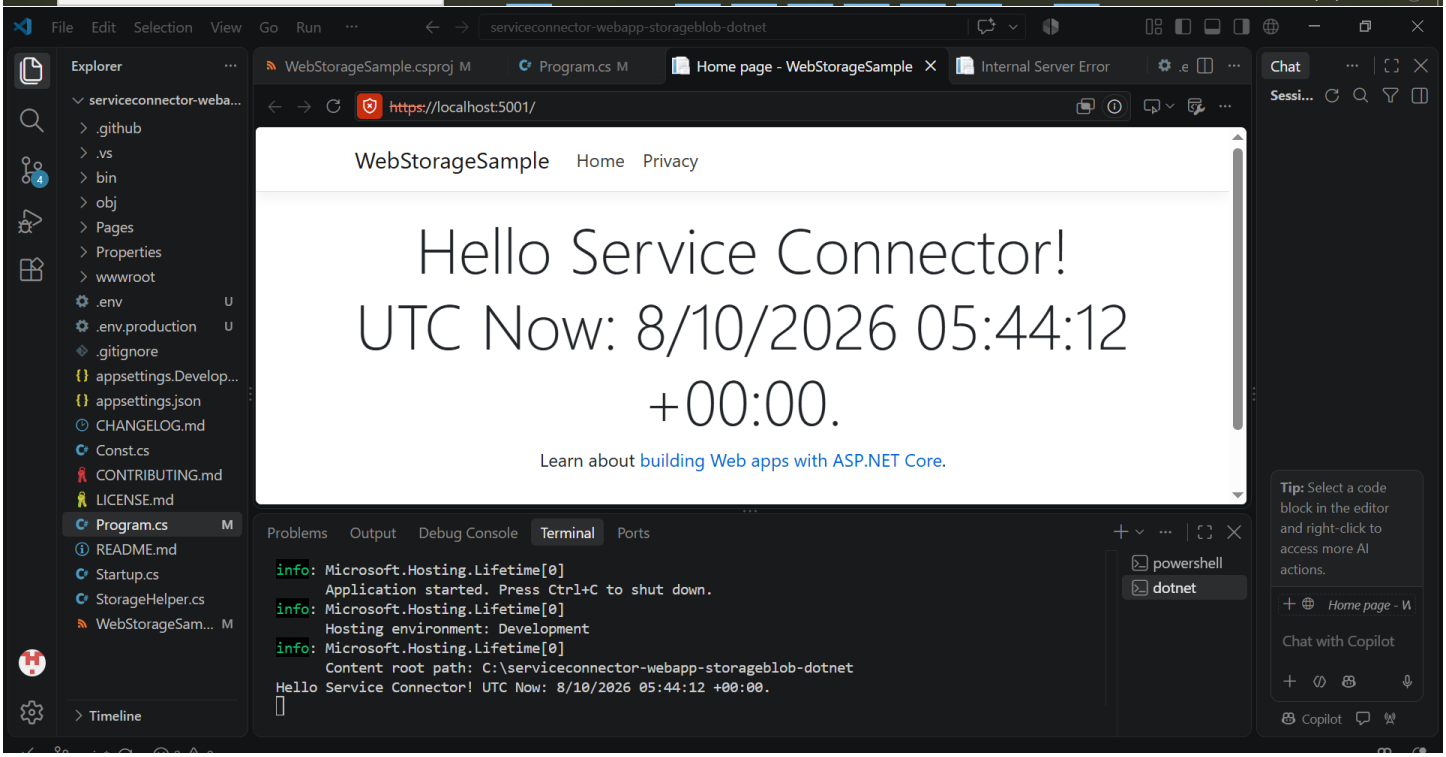
Revisaría la identidad administrada del App Service, que esté habilitada y bien configurada; el RBAC sobre Blob Storage, que tenga rol Storage Blob Data Contributor sobre la Storage Account o el contenedor correcto; el código de autenticación, que use DefaultAzureCredential y el scope correcto; el firewall o redes de la Storage Account, si permite acceso desde el App Service mediante VNet Integration, Service Endpoint o Private Endpoint; si usa SAS o Storage Account Key, que no haya expirado, tenga permisos de escritura y la connection string sea correcta; si la subida es directa desde el navegador, revisar CORS y permisos de la SAS; y finalmente los logs del App Service y de Storage para ver el mensaje exacto de AuthorizationFailure o AuthenticationFailed.

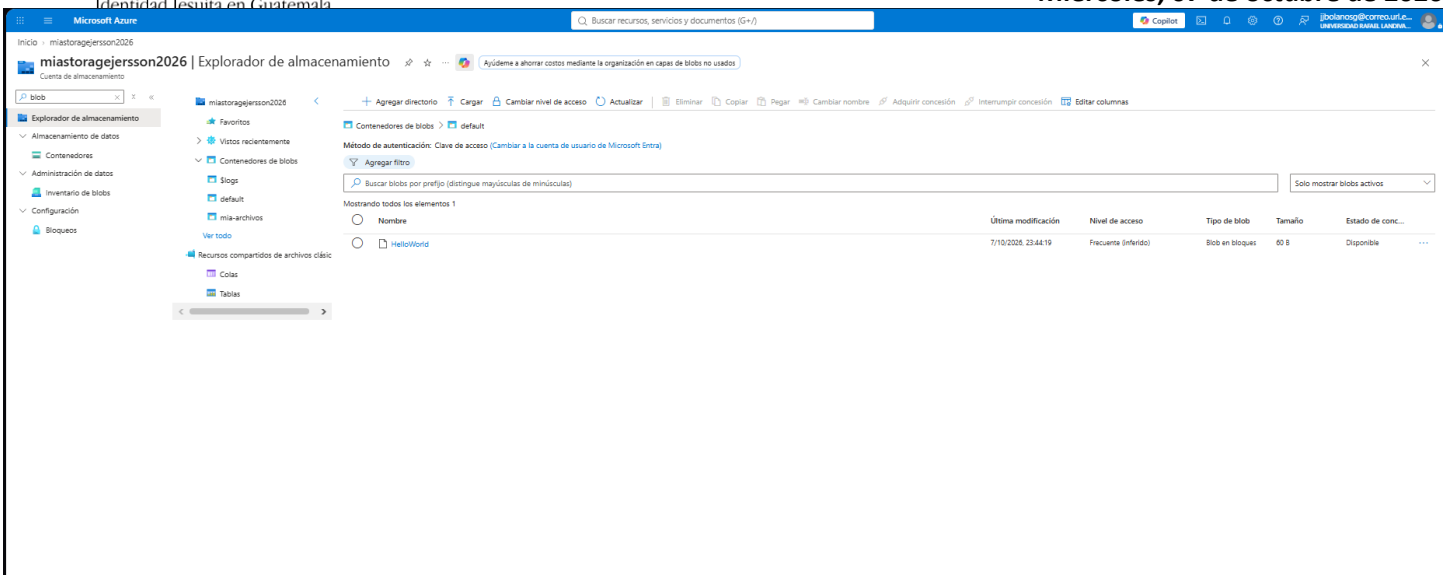
Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-14/lab1/ y coloque en ella un PDF con la solución del laboratorio.
Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.
Enviar link de github a plataforma de la universidad.




Seleccionar C:\serviceconnector-webapp-storageblob-dotnet\bin\Debug\net10.0\WebStorageSample.exe
System.IO.DirectoryNotFoundException: Could not find a part of the path 'C:\Users\Jersson\AppData\Local\ASP.NET\DataProtection-Keys\46f193fc-ea21-410a-8d96-6e2a88f8fa1f.tmp'.
at System.IO.FileSystem.DeleteFile(String fullPath)
at Microsoft.AspNetCore.DataProtection.Repositories.FileSystemXmlRepository.StoreElementCore(XElement element, String filename)
at Microsoft.AspNetCore.DataProtection.Repositories.FileSystemXmlRepository.StoreElement(XElement element, String friendlyName)
at Microsoft.AspNetCore.DataProtection.KeyManagement.XmlKeyManager.Microsoft.AspNetCore.DataProtection.KeyManagement.Internal.IInternalXmlKeyManager.CreateNewKey(Guid keyId, DateTimeOffset creationDate, DateTimeOffset activationDate, DateTimeOffset expirationDate)
at Microsoft.AspNetCore.DataProtection.KeyManagement.XmlKeyManager.CreateNewKey(DateTimeOffset activationDate, DateTimeOffset expirationDate)
at Microsoft.AspNetCore.DataProtection.KeyManagement.KeyRingProvider.CreateCacheableKeyRingCore(DateTimeOffset now, IKey keyJustAdded)
at Microsoft.AspNetCore.DataProtection.KeyManagement.KeyRingProvider.GetCurrentKeyRingCoreNew(DateTime utcNow, Boolean forceRefresh)
warn: Microsoft.AspNetCore.Server.Kestrel.Core.KestrelServer[8]
The ASP.NET Core developer certificate is not trusted. For information about trusting the ASP.NET Core developer certificate, see https://aka.ms/aspnet/https-trust-dev-cert
info: Microsoft.Hosting.Lifetime[14]
Now listening on: https://localhost:5001
info: Microsoft.Hosting.Lifetime[14]
Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
Content root path: C:\serviceconnector-webapp-storageblob-dotnet


WebStorageSample
Home
Privacy
Hello Service Connector!
UTC Now: 8/10/2026 05:44:12 +00:00.
Learn about building Web apps with ASP.NET Core.
info: Microsoft.Hosting.Lifetime[0]
Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
Content root path: C:\serviceconnector-webapp-storageblob-dotnet
Hello Service Connector! UTC Now: 8/10/2026 05:44:12 +00:00.


Microsoft Azure
Buscar recursos, servicios y documentos (G+/)
Copilot
Inicio › miastoragejersson2026
miastoragejersson2026 | Explorador de almacenamiento
Cuenta de almacenamiento
Ayúdeme a ahorrar costos mediante la organización en capas de blobs no usados
blob
Explorador de almacenamiento
Almacenamiento de datos
Contenedores
Administración de datos
Inventario de blobs
Configuración
Bloqueos
miastoragejersson2026
Favoritos
Vistos recientemente
Contenedores de blobs
$logs
default
mia-archivos
Ver todo
Recursos compartidos de archivos clásic
Colas
Tablas
Agregar directorio
Cargar
Cambiar nivel de acceso
Actualizar
Eliminar
Copiar
Pegar
Cambiar nombre
Adquirir concesión
Interrumpir concesión
Editar columnas
Contenedores de blobs
default
Método de autenticación: Clave de acceso (Cambiar a la cuenta de usuario de Microsoft Entra)
Agregar filtro
Buscar blobs por prefijo (distingue mayúsculas de minúsculas)
Solo mostrar blobs activos
Mostrando todos los elementos 1
Nombre
Última modificación
Nivel de acceso
Tipo de blob
Tamaño
Estado de conc...
HelloWorld
7/10/2026, 23:44:19
Frecuente (inferido)
Blob en bloques
60 B
Disponible